

4. Pass A and Pass B — in plain English

Audience: Charles, right before you re-run simulations
Read this after: docs 01–03 and the checklist self-test (hero PNG confirmed)
Read this before: opening `540_three_step_sim.ipynb` or running the Pass A / Pass B scripts
Companion (shorthand after this): `../../../sports/540_READ_ME_SIM.md`

Where you are in the story

You have already:

- Read what the **hero** is (real NCAA data: draft rate vs teammate pool quality).
- Separated **three layers** (describe the curve / tell the mechanism story / run a fake league).
- Confirmed you can say: environment $\neq$ score $\neq$ select.

Next job is not more theory. Next job is to **re-run two simulation experiments yourself** so the generative story is something *you* did, not something an agent left on disk.

Those two experiments are called **Pass A** and **Pass B**. The rest of this page explains them in full sentences. Only after that do we allow short labels like “ λ knockout” or “ ρ ablation.”

Are we only relying on empirical data?

No — not for Pass A and Pass B.

- The **hero** (Layer A) is **empirical**: real player-seasons, real draft outcomes. You are **not** rebuilding that now. You may look at the PNG/CSV as a fixed fact.
- Pass A and Pass B are **generative**: the computer **creates** a fake league of players and teams, applies rules, and then plots **who got selected** in that fake world.

So you are doing **two different jobs** in the project:

Job	Data	Purpose
Hero	Real MBB panel	“Here is the stylized fact in the world.”
Pass A / Pass B	Simulated league	“Here is whether a <i>rule</i> can bend a curve when we control the ingredients.”

Are we actually creating simulated data? How?

Yes.

In code, roughly:

1. Draw a large set of **synthetic players**, each with an ability number **A_i** (drawn from a distribution in the config — not taken from ESPN rows for these runs).
2. Draw a set of **team targets T_j** (ideal “team quality” levels).
3. **Assign** players to teams (form rosters) using a rule you choose (soft matching with assortativity **p**, or a hard sort-and-chop benchmark).
4. On each fake roster, compute leave-one-out pool statistics (quality and congestion), the same *kinds* of objects we use conceptually for the hero.
5. **Score** every player with a ranking formula **S_i**.
6. **Select** the top **K** players by that score (winner rule).
7. Bin players by simulated leave-one-out pool quality and plot the **fraction selected** in each bin.

That whole recipe is a **data-generating process** (a computerized thought experiment). It is **not** a regression on the real panel.

What code is used to simulate?

Think in **three layers of files** (do not merge them in your head):

1. **Engines (the library that does the math)**
 - `sports/tier1_pool_assignment.py` — draw abilities, assign to teams, compute LOO pools, score, pick top K
 - `sports/tier1_generative_edu.py` — run a league and build the binned selection table
 - `sports/tier1_sim_config.py` — default knobs (how many teams, roster size, ρ default, K, etc.)
2. **Bundles (the experiments you actually run)**
 - Pass A: `sports/scripts/hero_model_reset_bundle.py`
 - Pass B: `sports/scripts/540_rho_ablation_bundle.py`
3. **Notebook (thin remote control / display — not the full DGP)**
 - `sports/540_three_step_sim.ipynb`
 - This notebook mostly points at export folders and can optionally call the two scripts.
 - **If you want to understand the experiment, read this page + run the scripts.** Do not expect the notebook alone to teach the DGP.

Spec pointer after this prose: `sports/540_READ_ME_SIM.md`.

How are we forming group assignments?

Assignment = who sits on which fake team.

Default story (**soft assignment**):

- Each team has a target quality T_j .
- Each player has ability A_i .
- Players are placed on teams with probabilities that prefer teams whose T_j is close to A_i , but not perfectly — so rosters **overlap** in talent the way real college teams do (many teams' [min, max] talent windows stack on the same ability axis).
- The user-facing knob for "how assortative is that match?" is ρ (rho):
 - ρ near 0 $\rightarrow$ lots of mixing (weaker players can land on strong targets and vice versa).
 - ρ higher $\rightarrow$ sharper matching to the nearest target.
 - A fixed scale σ stays in the background; you do not need to re-invent it to run Pass A/B.

There is also a **sort-and-chop** path: sort everyone by ability and slice into equal groups. That creates **almost no talent overlap** between teams. It is a **benchmark**, not " ρ equals infinity," and not how real NCAA rosters look.

Are we trying to mimic empirical real data as we simulate?

Partly — for realism of *pools*, not for copying the hero bar heights.

What we *do* try to respect (from earlier 530 forensics):

- Real team talent windows **overlap** a lot.
- Typical within-roster spread is on the order of $\sim 0.8 z$ (not every team is a tiny talent clump).

Soft assignment with moderate ρ exists so the fake league is not a cartoon partition of the talent spectrum.

What we **do not** require for v1:

- Matching the hero's draft rate in every ventile bin.
- Copying every real college team's mean into the simulation.
- Proving the NBA literally uses our formula.

So: **mimic the *shape of roster formation* enough that the experiment is fair; do not claim the sim *is* the hero.**

The three-step pipeline (same spine for both passes)

Every generative run uses the same order:

1. **Assign** — put players on teams (ρ lives here).
2. **Score** — compute S_i for each player (λ / congestion weight lives here).
3. **Select** — given the scores, pick winners (v1: **top K**).

Important: "Score" and "select" are different steps. Turning congestion on or off changes the **score**. Changing ρ changes **who shares a roster**. Top K is the **winner rule**.

Pass A — what it is, in sentences

Name: Pass A (also called the λ **knockout** once you know what that means).

Question Pass A answers:

If we build a fake league and always pick the top K by score, does putting congestion into the ranking formula change who gets selected — compared with ranking on ability alone?

What stays the same in Pass A:

- How we think about forming teams (assignment settings from the config).
- The winner rule: still **top K**.
- The idea of binning on leave-one-out pool quality and plotting selection rates.

What changes in Pass A (the knockout):

- **Arm 1 — talent-only:** each player's score is basically $S_i = A_i$. Congestion does **not** enter the ranking. This is the " $\lambda = 0$ " story: ability alone.
- **Arm 2 — congestion in the score:** each player's score penalizes leave-one-out viable-peer congestion, conceptually $S_i = A_i - \lambda \cdot L_C$ (in code, a weight w on smooth crowding). Congestion **does** enter the ranking.

What you should see (qualitatively):

- Talent-only: selection rate tends to **rise** toward better pools (more talent concentrated there) — **no** elite congestion story.
- Congestion-in-score: the **top** pool-quality bins get **compressed** relative to talent-only — congestion in the score did real work.

What Pass A does *not* prove:

- That the sim matches the hero bar-for-bar.
- That NBA teams literally maximize S_i .
- That assignment assortativity (ρ) is irrelevant (that is Pass B's question).

What you run for Pass A:

```
python sports/scripts/hero_model_reset_bundle.py
```

Where outputs go:

sports/datasets/mbb/exports_inverted_u_v0/alex_side_by_side_v0/

(side-by-side PNG, knockout CSVs, summary text)

Pass B — what it is, in sentences

Name: Pass B (also called the **p ablation** once you know what that means).

Question Pass B answers:

Holding the scoring rule and the top-K winner rule fixed, does changing how assortatively we assign players to teams change the binned selection curve?

- What stays the same in Pass B:**
- The **score** formula (congestion-in-score version, fixed weight).
 - The **winner rule** (top K).
 - The same synthetic abilities and team targets across arms (same “talent deck,” different seating charts).

- What changes in Pass B:**
- **p low** — more mixing when forming teams.
 - **p moderate** — middle assortativity.
 - **p high** — sharper ability–team matching.
 - **sort-and-chop** — hard non-overlapping slices (benchmark).

- What Pass B is for:**
- To see whether **roster formation / sorting** moves the readout when the **score** story is held fixed.
 - In the **full** scientific story, assignment matters (step 1).
 - Pass B is **not** the minimal proof that congestion belongs **in the score**. That proof is Pass A.

- What Pass B does *not* prove:**
- That the NBA uses p.
 - That you must calibrate p to the hero before talking to Alex.
 - That sort-and-chop is “ $p \rightarrow \infty$.”

What you run for Pass B:

python sports/scripts/540_rho_ablation_bundle.py

Where outputs go:

sports/datasets/mbb/exports_inverted_u_v0/alex_rho_ablation_v0/

How Pass A and Pass B sit together (still in sentences)

- **Hero** = what happened in **real** data.
 - **Pass A** = in a **fake** league, does congestion in the **score** matter (yes/no knockout)?
 - **Pass B** = in a **fake** league, with that score held fixed, does **assignment assortativity** move the picture?
- You can explain Alex’s minimal generative claim with **Pass A** alone. Pass B is the honest follow-up about sorting / grouping.

After you understand this page

1. Optional skim: [sports/540_READ_ME_SIM.md](#) (now the shorthand tables will make sense).
 2. Return to [CHARLES_CHECKLIST.md](#) §3 and run Pass A.
 3. Then §4 and run Pass B.
 4. Optional: open [sports/540_three_step_sim.ipynb](#) only to display PNGs or re-call the scripts.
- If shorthand starts flooding you again: close other tabs and re-read **this** document from the top.